

Module 3: Loops and Methods

Add the information to demonstrate the following objectives and organize with headings:

Increments

Prefix	Postfix
<pre>int prefix = 5; System.out.println(prefix); System.out.println(++prefix);</pre> Console: 5 6 Changes the one before it	<pre>int prefix = 5; System.out.println(prefix++); System.out.println(prefix);</pre> Console: 5 6 Changes the one after it

Decrements

Prefix	Postfix
<pre>int prefix = 5; System.out.println(prefix); System.out.println(--prefix);</pre> Console: 5 4 Changes the one before it	<pre>int prefix = 5; System.out.println(prefix--); System.out.println(prefix);</pre> Console: 5 4 Changes the one after it

While	Do While
<pre>//while exponent negative or zero while (exponent <= 0) { //while exponent negative or zero System.out.println("Error: Exponent inputted is negative or zero."); System.out.print("Enter a positive number for an exponent: "); exponent = keyboardInput.nextInt(); }</pre> A while loop is while something is meeting the parameter, the contents within it will continue to execute.	<pre>do { // Prompt for input System.out.print("Enter a positive number for an exponent: "); // initializes exponent exponent = keyboardInput.nextInt(); if (exponent <= 0); { System.out.println("Error: Exponent inputted is negative or zero."); } } while (exponent <= 0);</pre> Do while, is that the while will always execute once. As long as the parameter of while is met it will continue looping.
For	
<pre>/* * count begins at 1, because the exponent CANNOT be 0. As long as count is less * than the exponent, every time it loops, it will count up. */ for (int count = 1; count < exponent; ++count) { // will multiply the base number by its original value until count is equal to // exponent base = base * baseMult; }</pre>	

Sentinel Values	Boolean Flags
<pre>while (exponent <= 0)</pre> 0 is the parameter in which the loop stops. It doesn't have to be an int, it can just have a readable value.	<pre>boolean loggedIn = false;</pre> Boolean flags are used to track whether a condition has been met or not in control structures. Think of it like a light switch. Usually changed by an if statement for conditional purposes.

Scope

Control Structure	Method
<pre>for (int attempts = 0; attempts <= 3; ++attempts) { boolean loggedIn = false; } attempts = 10; (ERROR)</pre>	<pre>public static void main(String[] args) { Scanner keyboardInput = new Scanner(System.in); System.out.print("What is your max?: "); int max = keyboardInput.nextInt(); } // end of userMax // max is now defined as inputtedMax public static int checkMax(int inputtedMax) { while (inputtedMax <= 0) { Scanner keyboardInput = new Scanner(System.in); max = keyboardInput.nextInt(); keyboardInput.close(); } return inputtedMax; } // end of checkMax</pre>
The integer attempt is within the parameters of the for loop, meaning it is ONLY within the scope of the for loop. The boolean loggedIn is also only within the scope of the for loop, because it is declared within the BRACES.	All variables and datatypes must be passed to the method before use via the parameter. Max is declared outside of the checkMax method, meaning that it is not in the SCOPE of the new method. keyboardInput is also declared outside of checkMax, so the Scanner must be built AGAIN.

Nested Control Structures/Loops

```
do
{
// the number of attempts begins at 0, there are no more than 3 attempts allowed
for (int attempts = 0; attempts <= 3; ++attempts)
{
System.out.print("Input username: ");
inputedUsername = keyboardInput.next();
// initialized inputedPassword
System.out.print("Input password: ");
inputedPassword = keyboardInput.next();
loggedIn = false;
// if user name is correct
if (USERNAME_ACTUAL.equals(inputedUsername))
{
// if both user name and password are correct
if (PASSWORD_ACTUAL.equals(inputedPassword))
{
// LOGGEDIN IS NOT ASSIGNED HERE, this outcome
//allows the user
// to pass the while loop
System.out.print("Login successful!");
}
// if ONLY the USER NAME is correct
else
{
System.out.println("Incorrect password");
// if the password is incorrect, logged in is false
}
}
// if user name is incorrect
else
{
// if ONLY the password is correct
if (PASSWORD_ACTUAL.equals(inputedPassword))
{
System.out.println("Incorrect username");
}
// if BOTH are incorrect
else
{
System.out.println("Both user+pass incorrect.");
}
}
}
} // end of for loop
} // end of do
// as long as this is false, it will do
while (loggedIn == true);
```

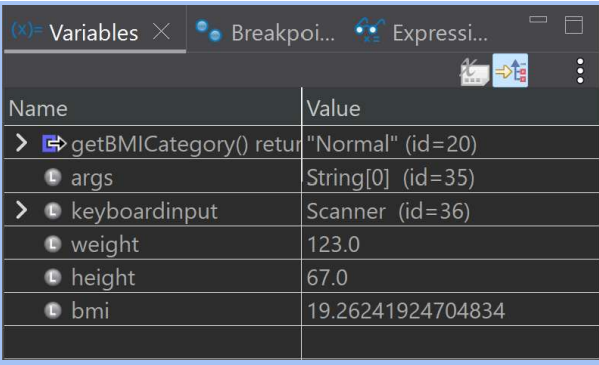
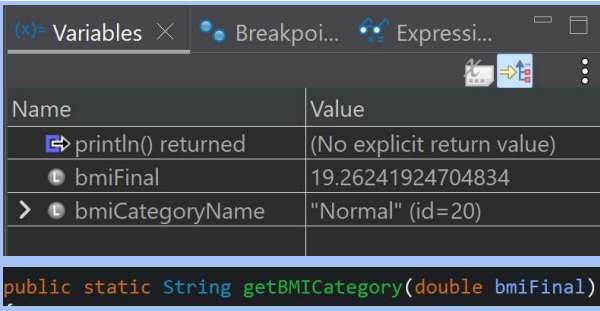
This example holds an if else statement, inside of another if else statement, inside of a for loop, inside of a do while loop.

What this does is:

1. It does the things within the do-while as long as the while condition is met
2. The for loop is allowed to loop a max

Methods

Write/Call				Pass/Returning
	Return	Name	(Parameter)	Remember to assign variables in other methods after a different one has been called.
static	int	name	int	
	void		string	
	double		int/double	
<pre>public static void main(String[] args) { printSummary(); } public static void printSummary() { }</pre>				<pre>public static void main(String[] args) { getPositiveDouble(keyboardinput, weight); } public static double getPositiveDouble(Scanner input, double bmiInput) { bmiInput = input.nextDouble(); return bmiInput; }</pre>

Trace Memory Methods																							
 Variables window showing state before <code>getBMICategory()</code> call: <table border="1"> <thead> <tr> <th>Name</th> <th>Value</th> </tr> </thead> <tbody> <tr> <td><code>getBMICategory() return</code></td> <td>"Normal" (id=20)</td> </tr> <tr> <td><code>args</code></td> <td>String[] (id=35)</td> </tr> <tr> <td><code>keyboardinput</code></td> <td>Scanner (id=36)</td> </tr> <tr> <td><code>weight</code></td> <td>123.0</td> </tr> <tr> <td><code>height</code></td> <td>67.0</td> </tr> <tr> <td><code>bmi</code></td> <td>19.26241924704834</td> </tr> </tbody> </table> Code snippet: <code>getBMICategory(bmi);</code>	Name	Value	<code>getBMICategory() return</code>	"Normal" (id=20)	<code>args</code>	String[] (id=35)	<code>keyboardinput</code>	Scanner (id=36)	<code>weight</code>	123.0	<code>height</code>	67.0	<code>bmi</code>	19.26241924704834	 Variables window showing state after <code>getBMICategory()</code> call: <table border="1"> <thead> <tr> <th>Name</th> <th>Value</th> </tr> </thead> <tbody> <tr> <td><code>println() returned</code></td> <td>(No explicit return value)</td> </tr> <tr> <td><code>bmiFinal</code></td> <td>19.26241924704834</td> </tr> <tr> <td><code>bmiCategoryName</code></td> <td>"Normal" (id=20)</td> </tr> </tbody> </table> Code snippet: <code>public static String getBMICategory(double bmiFinal)</code>	Name	Value	<code>println() returned</code>	(No explicit return value)	<code>bmiFinal</code>	19.26241924704834	<code>bmiCategoryName</code>	"Normal" (id=20)
Name	Value																						
<code>getBMICategory() return</code>	"Normal" (id=20)																						
<code>args</code>	String[] (id=35)																						
<code>keyboardinput</code>	Scanner (id=36)																						
<code>weight</code>	123.0																						
<code>height</code>	67.0																						
<code>bmi</code>	19.26241924704834																						
Name	Value																						
<code>println() returned</code>	(No explicit return value)																						
<code>bmiFinal</code>	19.26241924704834																						
<code>bmiCategoryName</code>	"Normal" (id=20)																						
Bmi is the value of a double When you want to set the value returned from the method, set whatever variable you want equal to method returned	The double for the new method is defined in the parameters. They both share the same value																						